

ISLP — Conceptual Question #1

K-Means Clustering: Equation (12.18) and Monotonic Descent

Doug Perez

School of Engineering & Technology, National University

DDS 8555 -- Predictive Analytics

Dr. Javier Leon

August 20, 2026

Companion deliverable: [PerezDDDS8555-7.pdf](#)

This notebook accompanies the written proof for Conceptual Question #1 on page 552 of *An Introduction to Statistical Learning: With Applications in Python* (James et al., 2023). It numerically verifies Equation (12.18) and illustrates why Algorithm 12.2 cannot increase the objective in Equation (12.17).

Question and analytical result

For cluster C_k , let $n_k = |C_k|$ and let

$$\bar{x}_{kj} = \frac{1}{n_k} \sum_{i \in C_k} x_{ij}.$$

Equation (12.18) states that

$$\frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2 = 2 \sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - \bar{x}_{kj})^2.$$

The companion PDF document proves this identity algebraically. The identity shows that the pairwise within-cluster objective in Equation (12.17) is exactly twice the ordinary within-cluster sum of squares (WCSS). Therefore, the two objectives have the same minimizers.

Algorithm 12.2 then alternates between two conditional minimizations:

1. **Assignment step:** With the centroids fixed, assign every observation to its nearest centroid.

2. **Centroid-update step:** With the assignments fixed, replace every centroid by its cluster mean.

Each step can decrease or preserve WCSS; neither step can increase it. The objective is therefore monotonically non-increasing across iterations (James et al., 2023). This alternating least-squares structure is also associated with the classical algorithm described by Lloyd (1982).

1. Setup

Kernel compatibility

This notebook requires only NumPy. A fixed random seed is used so that all numerical results are reproducible.

```
In [1]: import numpy as np

SEED = 8555
rng = np.random.default_rng(SEED)

def format_value(value):
    """Format values consistently without third-party table packages."""
    if isinstance(value, (float, np.floating)):
        return "-" if np.isnan(value) else f"{value:,.8f}"
    return str(value)

def print_table(rows, columns):
    """Print a compact, dependency-free table."""
    text_rows = [
        [format_value(row[column]) for column in columns]
        for row in rows
    ]

    widths = [
        max(
            len(column),
            *(len(row[index]) for row in text_rows),
        )
        for index, column in enumerate(columns)
    ]

    header = " | ".join(
        column.ljust(widths[index])
        for index, column in enumerate(columns)
    )
    separator = "--+--".join(
        "-" * width
        for width in widths
    )
```

```

print(header)
print(separator)

for row in text_rows:
    formatted_row = " | ".join(
        value.ljust(widths[index])
        for index, value in enumerate(row)
    )
    print(formatted_row)

print(f"NumPy version: {np.__version__}")
print(f"Random seed: {SEED}")

```

NumPy version: 2.5.1

Random seed: 8555

2. Functions corresponding to Equations (12.17) and (12.18)

The first function computes the left side of Equation (12.18) by summing squared distances over all **ordered** pairs within a cluster and dividing by the cluster size. The second computes the right side using the cluster mean. The remaining functions extend these quantities to an entire partition.

```

In [2]: def cluster_pairwise_objective(X_cluster):
    """Left side of Equation (12.18) for one nonempty cluster."""
    X_cluster = np.asarray(X_cluster, dtype=float)
    if X_cluster.ndim != 2 or len(X_cluster) == 0:
        raise ValueError("X_cluster must be a nonempty two-dimensional array.")
    differences = X_cluster[:, None, :] - X_cluster[None, :, :]
    return np.square(differences).sum() / len(X_cluster)

def cluster_twice_wcss(X_cluster):
    """Right side of Equation (12.18) for one nonempty cluster."""
    X_cluster = np.asarray(X_cluster, dtype=float)
    if X_cluster.ndim != 2 or len(X_cluster) == 0:
        raise ValueError("X_cluster must be a nonempty two-dimensional array.")
    mean = X_cluster.mean(axis=0)
    return 2.0 * np.square(X_cluster - mean).sum()

def centroids_from_labels(X, labels, K):
    """Return the mean vector for each of K nonempty clusters."""
    centroids = []
    for k in range(K):
        members = X[labels == k]
        if len(members) == 0:
            raise ValueError(f"Cluster {k} is empty.")
        centroids.append(members.mean(axis=0))
    return np.vstack(centroids)

```

```

def wcss(X, labels, centroids):
    """Within-cluster sum of squared distances to supplied centroids."""
    return np.square(X - centroids[labels]).sum()

def partition_pairwise_objective(X, labels, K):
    """Equation (12.17) for a complete K-cluster partition."""
    return sum(cluster_pairwise_objective(X[labels == k]) for k in range(K))

```

3. Numerical verification of Equation (12.18)

The following fixed two-dimensional observations and cluster labels are constructed solely for verification. No external dataset is required.

```

In [3]: X_check = np.array([
    [1.0, 1.0],
    [1.5, 2.0],
    [2.0, 1.5],
    [7.0, 7.5],
    [8.0, 7.0],
    [7.5, 8.5],
    [12.0, 2.0],
    [13.0, 3.0],
    ], dtype=float)

labels_check = np.array([0, 0, 0, 1, 1, 1, 2, 2])
K_check = 3

identity_rows = []
for k in range(K_check):
    members = X_check[labels_check == k]
    left = cluster_pairwise_objective(members)
    right = cluster_twice_wcss(members)
    identity_rows.append({
        "Cluster": k + 1,
        "n_k": len(members),
        "Pairwise expression (left)": left,
        "Twice WCSS (right)": right,
        "Absolute difference": abs(left - right),
    })

identity_columns = [
    "Cluster",
    "n_k",
    "Pairwise expression (left)",
    "Twice WCSS (right)",
    "Absolute difference",
]

print_table(identity_rows, identity_columns)

max_identity_error = max(row["Absolute difference"] for row in identity_rows)
assert np.isclose(max_identity_error, 0.0, atol=1e-12)

```

```
print(f"Maximum absolute difference: {max_identity_error:.3e}")
print("Equation (12.18) is numerically verified for every cluster.")
```

Cluster	n_k	Pairwise expression (left)	Twice WCSS (right)	Absolute difference
1	3	2.00000000	2.00000000	0.00000000
2	3	3.33333333	3.33333333	0.00000000
3	2	2.00000000	2.00000000	0.00000000

Maximum absolute difference: 0.000e+00

Equation (12.18) is numerically verified for every cluster.

Interpretation

For each cluster, the pairwise expression on the left side of Equation (12.18) equals twice the sum of squared distances from the observations to their cluster mean. Any residual displayed in the final column is limited to floating-point rounding. This computation verifies the identity for the example; the general result is established by the algebraic proof in the companion PDF document.

4. A transparent implementation of Algorithm 12.2

The next function implements the two K-means steps directly with NumPy. It records the objective immediately after the assignment step and again after the centroid-update step. It also records Equation (12.17) after each complete iteration.

```
In [4]: def assign_to_nearest_centroid(X, centroids):
        """Assign each observation to its closest centroid."""
        squared_distances = np.square(X[:, None, :] - centroids[None, :, :]).sum(axis=2)
        return squared_distances.argmin(axis=1)

def run_kmeans_with_trace(X, initial_centroids, max_iter=100, tolerance=1e-12):
    """Run K-means and retain objectives after both alternating steps."""
    X = np.asarray(X, dtype=float)
    centroids = np.asarray(initial_centroids, dtype=float).copy()
    K = len(centroids)
    trace = []
    previous_completed_wcss = None

    for iteration in range(1, max_iter + 1):
        # Step 1: minimize over assignments while holding centroids fixed.
        labels = assign_to_nearest_centroid(X, centroids)
        assignment_wcss = wcss(X, labels, centroids)

        # Step 2: minimize over centroids while holding assignments fixed.
        updated_centroids = centroids_from_labels(X, labels, K)
        updated_wcss = wcss(X, labels, updated_centroids)
        pairwise_objective = partition_pairwise_objective(X, labels, K)

        trace.append({
            "Iteration": iteration,
            "WCSS after assignment": assignment_wcss,
```

```

        "WCSS after centroid update": updated_wcss,
        "Equation (12.17) objective": pairwise_objective,
        "Update reduction": assignment_wcss - updated_wcss,
        "Change from prior completed WCSS": (
            np.nan if previous_completed_wcss is None
            else previous_completed_wcss - updated_wcss
        ),
    })

    # Equation (12.18): the pairwise objective equals twice updated WCSS.
    assert np.isclose(pairwise_objective, 2.0 * updated_wcss, atol=1e-10)
    # Replacing each centroid by its cluster mean cannot increase WCSS.
    assert updated_wcss <= assignment_wcss + tolerance
    # A complete iteration cannot increase the previous completed WCSS.
    if previous_completed_wcss is not None:
        assert updated_wcss <= previous_completed_wcss + tolerance

    if np.allclose(updated_centroids, centroids, atol=tolerance, rtol=0.0):
        centroids = updated_centroids
        break

    previous_completed_wcss = updated_wcss
    centroids = updated_centroids

return labels, centroids, trace

```

5. Demonstration on deterministic synthetic data

Three compact point clouds are generated with a fixed random seed. The initial centroids are deliberately selected from the same initial point cloud, rather than one from each eventual group. This reproducible poor initialization makes the objective reductions visible before the algorithm stabilizes.

```

In [5]: true_centers = np.array([
        [-4.0, -1.0],
        [0.5, 4.5],
        [5.0, -0.5],
    ])

X_demo = np.vstack([
    rng.normal(loc=center, scale=[1.15, 0.90], size=(24, 2))
    for center in true_centers
])

# Fixed, reproducible starting points chosen from the generated observations.
initial_indices = np.array([0, 1, 2])
initial_centroids = X_demo[initial_indices]

final_labels, final_centroids, objective_trace = run_kmeans_with_trace(
    X_demo,
    initial_centroids,
)

```

```

objective_display_rows = [
    {
        "Iter.": row["Iteration"],
        "Assignment WCSS": row["WCSS after assignment"],
        "Updated WCSS": row["WCSS after centroid update"],
        "Eq. (12.17)": row["Equation (12.17) objective"],
    }
    for row in objective_trace
]

print("Objective values by iteration")
print_table(
    objective_display_rows,
    [
        "Iter.",
        "Assignment WCSS",
        "Updated WCSS",
        "Eq. (12.17)",
    ],
)

reduction_display_rows = [
    {
        "Iter.": row["Iteration"],
        "Update reduction": row["Update reduction"],
        "Prior reduction": (
            row["Change from prior completed WCSS"]
        ),
    }
    for row in objective_trace
]

print("\nReductions by iteration")
print_table(
    reduction_display_rows,
    [
        "Iter.",
        "Update reduction",
        "Prior reduction",
    ],
)

print("\nFinal centroids:")
centroid_rows = [
    {"Cluster": index + 1, "Feature 1": center[0], "Feature 2": center[1]}
    for index, center in enumerate(final_centroids)
]
print_table(centroid_rows, ["Cluster", "Feature 1", "Feature 2"])
print(f"Converged after {len(objective_trace)} iterations.")

```

Objective values by iteration

Iter.	Assignment WCSS	Updated WCSS	Eq. (12.17)
1	2,703.50631354	619.94046709	1,239.88093418
2	227.51944276	108.34406776	216.68813553
3	108.34406776	108.34406776	216.68813553

Reductions by iteration

Iter.	Update reduction	Prior reduction
1	2,083.56584645	—
2	119.17537500	511.59639933
3	0.00000000	0.00000000

Final centroids:

Cluster	Feature 1	Feature 2
1	5.17312264	-0.50905961
2	0.23855513	4.69629965
3	-3.99764987	-1.03954574

Converged after 3 iterations.

6. Explicit monotonicity checks

The assertions below test all numerical implications used in part (b):

- the centroid update does not increase WCSS within an iteration;
- the completed WCSS does not increase from one iteration to the next; and
- Equation (12.17) equals twice the completed WCSS.

```
In [6]: tol = 1e-10

update_is_nonincreasing = all(
    row["WCSS after centroid update"] <= row["WCSS after assignment"] + tol
    for row in objective_trace
)

completed_wcss = np.array([
    row["WCSS after centroid update"] for row in objective_trace
])
completed_is_nonincreasing = np.all(np.diff(completed_wcss) <= tol)

identity_holds = np.allclose(
    [row["Equation (12.17) objective"] for row in objective_trace],
    2.0 * completed_wcss,
    atol=tol,
)

checks = [
    {
        "Check": "Centroid-update WCSS never increases",
        "Verified": update_is_nonincreasing,
    },
    {
```



```

    "Check": "Completed WCSS never increases across iterations",
    "Verified": completed_is_nonincreasing,
},
{
    "Check": "Equation (12.17) equals twice WCSS",
    "Verified": identity_holds,
},
]

print_table(checks, ["Check", "Verified"])
assert all(row["Verified"] for row in checks)
print("All numerical checks passed.")

```

Check	Verified
Centroid-update WCSS never increases	True
Completed WCSS never increases across iterations	True
Equation (12.17) equals twice WCSS	True
All numerical checks passed.	

Objective reduction summary

```

In [7]: first_objective = objective_trace[0]["Equation (12.17) objective"]
final_objective = objective_trace[-1]["Equation (12.17) objective"]
absolute_reduction = first_objective - final_objective
percentage_reduction = 100.0 * absolute_reduction / first_objective

summary_rows = [{
    "Initial objective": first_objective,
    "Final objective": final_objective,
    "Absolute reduction": absolute_reduction,
    "Percentage reduction": percentage_reduction,
}]
summary_columns = [
    "Initial objective",
    "Final objective",
    "Absolute reduction",
    "Percentage reduction",
]

print_table(summary_rows, summary_columns)
print("The final repeated value confirms that the algorithm has stabilized.")

```

Initial objective	Final objective	Absolute reduction	Percentage reduction
1,239.88093418	216.68813553	1,023.19279865	82.52347225
The final repeated value confirms that the algorithm has stabilized.			

7. Findings and conclusion

The numerical results support the two claims established analytically in the companion PDF document.

First, the direct pairwise calculation and twice-WCSS calculation agree for every cluster, illustrating Equation (12.18). Second, the execution trace shows that replacing each centroid with its cluster mean does not increase WCSS and that the completed objective is monotonically non-increasing across iterations. The final equality between Equation (12.17) and twice WCSS connects the computational trace directly to the objective named in the question.

The numerical demonstration is not a substitute for the general proof. It verifies the result for reproducible examples and makes the two optimization steps observable. K-means converges to a stable partition, but the stable solution need not be the global minimum; different initial centroids can lead to different local solutions (James et al., 2023; Lloyd, 1982).

References

- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An introduction to statistical learning: With applications in Python*. Springer. <https://doi.org/10.1007/978-3-031-38747-0>
- Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137. <https://doi.org/10.1109/TIT.1982.1056489>